

1) Differentiate between pop and oop.

pop - procedure-oriented programming

oop - object-oriented programming.

→ focuses on function/procedure → focuses on object and classes.

→ Data is shared globally → Data is protected through encapsulation.

→ uses a top-down approach → uses a bottom-up approach

→ Less secure

→ More secure due to data hiding

→ ex:- c, pascal

→ ex:- Java, c++, python.

2) Explain the Basic Concepts of OOP.

→ Class:- A blueprint for creating objects.

→ objects:- An instance of a class with properties (data) and behaviors (methods)

→ Encapsulation: Wrapping data and method into a single unit and programme, restricting direct access.

→ Inheritance:- A class acquires properties and methods from another class.

→ Polymorphism:- One interface with multiple implementation.

→ Abstraction:- Hiding implementation details and showing only essential feature.

3) Develop simple Java programs using classes and objects.

```
class student {  
    String name;  
    int age;  
    void display() {  
        System.out.println("Name:" + name);  
        System.out.println("Age:" + age);  
    }  
    public static void main (String[] args) {  
        student s1 = new student ();  
        s1.name = "Rahul";  
        s1.age = 20;  
        s1.display();  
    }  
}
```

output:- Name: Rahul
Age: 20.

4) Apply operators and control statements:

Operators

→ Arithmetic: +, -, *, /, %

→ Relational: ==, !=, >, <, >=, <=

→ Logical: &&, ||, !

→ Assignment: =, +=, -=

2500032235

B. Jugal

```

public class controlExample {
    public static void main (String[] args)
    {
        int mark = 75;
        if (mark >= 50) {
            System.out.println("pass");
        } else {
            System.out.println("fail");
        }
        for (int i = 1; i <= 5; i++) {
            System.out.println(i);
        }
    }
}

```

5) Use Arrays and strings in java programs array example.

```

public class arrayExample {
    public static void main (String[] args) {
        int[] numbers = {10, 20, 30, 40};
        for (int num; numbers) {
            System.out.println(num);
        }
    }
}

```

```

public class stringExample {
    public static void main (String[] args) {
        String name = "java";
        System.out.println (name.toUpperCase());
        System.out.println (name.length());
    }
}

```

2500032-235

6. Write well-structured and Readable Java code.

Good coding practices include:

- use meaningful class, method, and variables names,
- follows Java naming convention (e.g. studentName, calculate()).
- Indent code consistently.
- Add comments where necessary.
- keep methods short and focused on a single task.
- avoid unnecessary repetition by reusing methods

Examples

```
public class calculator {  
    static int add (int a, int b) {  
        return a + b;  
    }  
    public static void main (String[] args) {  
        int result = add (10, 20);  
        System.out.println ("sum = " + result);  
    }  
}
```

SV
2/08/20

2500032235

B. Jugal